PORTFOLIO / MASTER BUILD BRIEF

Priyansh Singh

A complete, implementation-ready brief for a premium technical portfolio with liquid-flow visuals, responsive interactions, dual themes, and verified project information.

Design direction: **Liquid Intelligence**

Inspired by the clarity of Apple product experiences, the scientific polish of Google DeepMind, and original fluid motion - without copying their interfaces.

Prepared for use with Codex, Google AI Studio, or another coding agent that can inspect and modify a real project repository.

Includes profile information, public links, project inventory, and a detailed build-and-QA prompt.

Light + Dark Mode | Liquid Motion | Custom Cursor | Responsive | Hackathon Ready

Contents

1. Portfolio goals and creative direction
2. Profile details, education, skills, and credentials
3. Public profiles and contact information
4. Project inventory: deployed, in progress, and concepts
5. Master build prompt for your coding agent
6. Quality assurance and definition of done

> **Accuracy rule:** Publicly accessible work should be prioritised. Unfinished projects must be labelled honestly, missing URLs must not be invented, and the deliberately excluded gesture-related project must remain absent.

The goal is not simply to make a portfolio look impressive. It is to make it feel polished while ensuring every project claim, destination, and interaction can withstand a judge clicking through the site.

1. Portfolio Goals and Creative Direction

The portfolio should act as a digital introduction to Priyansh Singh: an ECE undergraduate who is learning by building practical systems across full-stack development, AI applications, cybersecurity, automation, and connected systems.

> **Primary objective:** Create an interactive, polished portfolio suitable for an in-person hackathon. Judges should be able to understand the person, inspect real projects, test the interface, and open the correct source code or live demo.

Design concept: Liquid Intelligence

- Deep charcoal and near-black surfaces in dark mode; warm white and neutral surfaces in light mode.
- Restrained electric blue, indigo, or cyan accents shared across both themes.
- Large, confident typography, clear hierarchy, spacious composition, precise alignment, and subtle depth.
- One memorable liquid surface or fluid visual in the hero, with lighter motion elsewhere.
- Custom cursor behaviour on suitable desktop devices, purposeful hover states, smooth transitions, and restrained scroll reveals.
- Original design language inspired by premium product websites and scientific visual polish, rather than copying any one site.

What to avoid

- Generic student-portfolio templates, excessive glassmorphism, noisy gradients, and random floating shapes everywhere.
- Constant animation that makes reading or clicking difficult.
- Fake metrics, made-up awards, fake client logos, or features the projects do not implement.
- Buttons without real actions, placeholder links, and demo links that point to localhost.
- Heavy graphics that cause stuttering, overflow, or make the site unusable when disabled.

2. Profile Information

Professional identity

Priyansh Singh
B.Tech undergraduate in Electronics and Communication Engineering (ECE) at Ajay Kumar Garg Engineering College (AKGEC), Ghaziabad, Uttar Pradesh, India.

Suggested positioning: *Exploring full-stack web development, AI-powered applications, cybersecurity, automation, and connected systems.*

Education

Degree	Bachelor of Technology (B.Tech), Electronics and Communication Engineering
Institution	Ajay Kumar Garg Engineering College (AKGEC), Ghaziabad
Dates	September 2024 - September 2028
School	Lions School, Mirzapur
Qualification	Intermediate and High School; Physics, Chemistry and Mathematics (PCM)
School dates	March 2019 - March 2023

Experience, leadership, and activities

- **Photographer, Photography and Media Club** - May 2025 to July 2025, Ghaziabad.
- **Phoenix Technical Society, ECE domain** - coordination and technical-event involvement, based on previously supplied details.
- **RoboVerse** - IoT-themed event organisation involving more than 200 students, based on previously supplied details. Present this as event leadership, not as a software product.
- **GirlScript Summer of Code 2026** - contributor/mentee acceptance. Describe the role accurately and do not imply a ranking or award.
- **CyberGeek CTF 2026** - 24-hour event associated with Geekhaven at IIIT Allahabad; more than 300 teams were reported in the supplied details. Do not claim a win or rank unless verified.

Technical skills

Programming	C, C++, Python, JavaScript
Web and backend	React, TypeScript, Vite, Flask, FastAPI, REST APIs, Node.js - use project evidence and current proficiency to describe these accurately.
Database and data tools	MySQL, NumPy, Pandas, Matplotlib
Security and networking	Cybersecurity fundamentals, networking basics, authorised network scanning, security dashboards
Embedded systems	Arduino, ESP32, motor drivers, IoT and robotics projects
Tools	Git, GitHub, VS Code, Vercel, Netlify

Additional interests	Responsible AI, multimodal prompting, photography and media

Skill list combines CV details and previously discussed project/tool experience. Do not present technologies as advanced skills merely because they appear in a dependency list. Keep proficiency statements honest.

Certifications listed in the CV

- Python (Basic)
- Introduction to Cybersecurity
- Cybersecurity Fundamentals
- Google Prompting Essentials Specialization
- Networking Basics

Link credentials only when the corresponding verifiable badge or certificate URL is available.

3. Profiles and Contact Links

GitHub	https://github.com/priyansh019
LinkedIn	https://www.linkedin.com/in/priyansh019
Credly	https://www.credly.com/users/priyansh19
LeetCode	https://leetcode.com/u/priyxnsh19/
HackerRank	https://www.hackerrank.com/profile/priyansh_singh01
Instagram (optional)	https://www.instagram.com/itspriyansh19
Email	priyansh.singh019@gmail.com

Keep professional profiles prominent and Instagram optional. Do not publicly display the mobile number by default. Do not invent coding streaks, profile statistics, domains, or credential URLs.

4. Project Inventory

Prioritise demonstrable work. The site should use a single typed data file for project descriptions, statuses, technologies, screenshots, and links so details can be corrected quickly before a hackathon.

01. SentinelPy - Network Security Scanner Deployed

A full-stack network-security platform. Repository documentation describes a React + Vite dashboard, FastAPI backend, and Python scanning engine, including port scanning, service fingerprinting, known-vulnerability matching, and risk classification.

Source: github.com/priyansh019/network-scanner
Live dashboard: network-scanner-2-umx2.onrender.com
API docs: network-scanner-1-p3wn.onrender.com/docs

Earlier conversations used the name NetSecure. Determine whether NetSecure and SentinelPy refer to the same project; do not create duplicate cards. Verify that the deployed app and features currently work before claiming them.

02. ClockWork - AI-Powered Everyday Tasks Deployed

An AI-powered application intended to assist with everyday tasks. The repository includes a TypeScript-based application and Gemini API integration. Verify detailed features in the current code.

Source: github.com/priyansh019/ClockWork
Live app: beclockwork.vercel.app

03. NovaBank - Banking Project Deployed

A banking application presented as a next-generation private-banking experience. The repository contains web-application code and banking-related Python files. Do not claim production banking, real transactions, regulatory compliance, or security properties without evidence.

Source: github.com/priyansh019/NovaBank
Live app: novabank-nu.vercel.app

04. LeetCode Solutions Ongoing practice

A C++ algorithms and data-structures practice repository. Treat as a coding-practice resource rather than a finished product.

Repository: github.com/priyansh019/Leetcode-Solutions
Profile: leetcode.com/u/priyxnsh19

05. NovaVoice - Calendar Bridge API

In progress / local prototype

A Flask API bridge connecting a frontend to Google Calendar. Previously discussed endpoints include `/`, `/api/health`, `/health`, `/api/book_meeting`, and `/api/check_availability`. The prototype covered calendar connectivity, availability checks, and booking requests.

No confirmed public deployment URL has been supplied. Never show localhost as a public demo. Add a repository link only when the real repository is known.

06. AI Workforce Management Platform

In progress

An AI-driven intelligent workforce-management platform discussed for a competition. Specific modules and completion claims must come from the actual brief or current code. No confirmed public repository or deployment URL has been supplied.

07. Local and Intercity Bus Route Explorer

Concept

An app concept for discovering local and intercity buses, intermediate stops, route options, and fares. Previously discussed examples include Noida Sector 34 to Meerut and a user-reported Mamura-to-Dasna trip costing Rs 25 by bus versus about Rs 50 by auto.

Those fares are user-reported examples, not verified live prices. No finished app or public URL has been established.

08. SAT-SA / SOC Assessment Analytics

Exploration

A supervisory analytics tool for SOC assessment discussed in connection with Smart India Hackathon problem statement PS 26157. Include as an active project only if a current codebase demonstrates that development is underway; otherwise omit it from the main public grid.

> **Explicit exclusion:** Do not include the gesture-related project, its source code, screenshots, links, or claims. Do not reintroduce it via automatic repository discovery.

5. Master Build Prompt

Copy the prompt below into Codex, Google AI Studio, or another coding agent that can inspect and modify the actual project repository.

Role and objective

Act as a senior frontend engineer, creative developer, motion designer, accessibility specialist, and QA engineer. Build a complete, polished, responsive, production-ready technical portfolio for Priyansh Singh, a B.Tech ECE undergraduate at AKGEC, Ghaziabad, India.

This portfolio is for an in-person hackathon, where judges and recruiters may open the site, interact with it, inspect projects, and follow source/demo links. It must communicate technical credibility, originality, and attention to detail. Do not stop at a static mockup; build the working application.

The final experience should have the quality of a carefully crafted digital product: the clarity of excellent Apple product pages, the scientific polish of Google DeepMind, and an original fluid interface. Use these as inspiration, not as interfaces to copy.

1. Non-negotiable requirements

1. Distinctive liquid-flow visual identity.
2. Fully functional light and dark themes.
3. Smooth scrolling and purposeful motion.
4. Custom cursor on suitable desktop pointer devices.
5. Interactive project cards and meaningful project details.
6. Excellent mobile, tablet, and desktop layouts.
7. Real profile links and project URLs from the supplied data.
8. Fast, polished first impression.
9. Accessible navigation and keyboard interactions.
10. Performance-conscious effects with fallbacks.
11. No fake projects, statistics, credentials, broken links, or misleading status labels.
12. A real QA process to find and fix errors before completion.

Do not claim the site is bug-free without testing it. Verify the implementation and report remaining limitations honestly.

2. Inspect the environment before coding

- Inspect the repository structure, package.json, dependencies, entry points, components, styles, configuration, README, environment variables, and existing functionality.
- Determine whether the project is an existing application to extend or a fresh build.
- Preserve working code and functionality unless there is a clear reason to change it.
- Inspect existing implementations before recreating features.
- Identify the available run, lint, type-check, test, and build commands.
- If GitHub access is available, inspect linked repositories before making detailed implementation claims.
- Never expose or commit API keys, tokens, or environment secrets.
- Make reasonable implementation decisions independently; ask only when a genuinely essential decision cannot be resolved.

3. Recommended technology

Prefer React, TypeScript, and Vite where compatible with the current project. Use Tailwind CSS or a well-structured CSS design system; Motion/Framer Motion for restrained animation; Lucide icons; and CSS transforms, SVG filters, Canvas, or lightweight WebGL for liquid effects.

Add dependencies only when they clearly improve the result. Do not replace a working stack unnecessarily. Adapt to the existing framework if needed.

4. Design system: Liquid Intelligence

Create a sophisticated, editorial, immersive interface with flowing surfaces, subtle light refraction, precise typography, soft gradients, spatial depth, and interactive motion.

- Use large, confident typography; a strong hierarchy; generous spacing; clean grids; rounded surfaces; restrained borders; subtle shadows; and purposeful negative space.
- Dark mode: deep charcoal/near-black backgrounds, off-white type, restrained electric blue/indigo/cyan accents, and subtle highlights.
- Light mode: warm white or light neutral backgrounds, deep charcoal typography, matching accents, precise borders, and restrained shadows.
- Build complete central design tokens so both themes are polished and independently considered; do not simply invert colours.
- Avoid excessive glassmorphism, random blobs, constant animation, overlarge headings that push content down, overloaded dashboards, and heavy gradients.
- Use designed icons rather than emoji as UI substitutes.

5. Liquid-flow visuals

Build one memorable liquid or fluid element in the hero, with subtle supporting motion elsewhere. Choose the most stable approach for the current stack: SVG displacement filters, a restrained Canvas particle/fluid field, a lightweight shader, a procedural gradient mesh, or pointer-driven distortion with spring easing.

- Movement must be organic, smooth, and coherent.
- Pointer interaction must not block links, text selection, or controls.
- Prevent horizontal overflow and unbounded animation work.
- Use requestAnimationFrame appropriately and clean up animation loops.
- React correctly to viewport changes and reduce work when the page is hidden where practical.
- Provide a static/CSS fallback when WebGL is unavailable.
- Respect `prefers-reduced-motion`.
- The website must stay fully usable when the effect is disabled.

6. Custom cursor and micro-interactions

On eligible desktop pointer devices, create a small primary cursor and a softly spring-following outer ring. Use restrained trailing, subtle hover scaling, contextual project/button states, optional short labels such as VIEW, magnetic movement on selected buttons, and a mild card spotlight.

- Do not force React renders on every mouse movement; use refs, animation frames, and transforms where appropriate.
- Clean up listeners and scheduled animation work.
- Use native cursors on touch and non-precise pointer devices.
- Never impair text inputs, selection, keyboard use, or click targets.
- The custom cursor must not capture events intended for the page.

7. Scrolling and motion

Use smooth, natural section transitions, a restrained scroll-progress indicator, a navigation bar that changes subtly on scroll, small reveal transitions, sparing staggered project entrances, refined hover transitions, and theme transitions.

Do not animate everything. Preserve native anchor navigation, keyboard scrolling, focus movement, browser history, and reduced-motion preferences. Prefer native scrolling if a smooth-scroll library would make mobile scrolling, sticky elements, or accessibility fragile.

8. Website structure

A. Navigation

Include wordmark/name, Home, Projects, About, Experience, Certifications, Contact, and a real theme toggle. Make the mobile menu work, close it after selection, support keyboard navigation, avoid placeholder href values, and account for sticky-header offsets.

B. Hero

Suggested headline: **“I turn curious ideas into useful systems.”**

Suggested supporting copy: “ECE undergraduate at AKGEC, exploring full-stack development, AI-powered applications, cybersecurity, and connected systems.”

Include name, technical identity, liquid visual, authentic introduction, Explore Projects CTA, Let’s Connect CTA, GitHub and LinkedIn links, and a CV download only if a real PDF asset exists. The hero should look good on desktop and mobile without a long mandatory animation.

C. Featured projects

Feature SentinelPy, ClockWork, and NovaBank first; then relevant in-progress work and coding practice. Each card should show title, one-sentence description, honest status, repository-grounded technologies, genuine screenshot or tasteful original preview, source link when available, live demo when available, and an accessible details action.

Do not fake screenshots of unavailable features, analytics, usage counts, customer logos, or performance claims. If a project has no public demo, say so by omission rather than inventing one.

D. Project filtering

Provide working filters such as All, AI & Automation, Web Applications, Cybersecurity, APIs & Systems, and Experiments & Practice, but show only categories that apply. Handle empty results, active states, keyboard use, and small screens.

E. Project details

Use an accessible modal, side panel, dedicated route, or similarly robust interaction. Include overview, problem, functionality, implementation approach, architecture, technology choices, current status, learning/reflection, repository, demo, and API docs where available. Hide unknown fields rather than inventing content.

For modals, support Escape, provide a visible close control, manage focus correctly, prevent background scrolling while open, restore focus on close, and fit small screens.

F. About

Describe an ECE undergraduate at AKGEC who enjoys building practical software and exploring web technologies, AI, cybersecurity, automation, and connected systems. Highlight curiosity, experimentation, teamwork, and continuous learning. Avoid claims such as “industry expert,” “senior engineer,” or “award-winning developer.”

G. Skills

Group programming, web/backend, security/networking, AI/data, embedded/electronics, and tools/platforms. Do not use invented proficiency percentages. Connect skills to real projects where possible and clearly separate learning technologies from established skills.

H. Experience and activities

Include the Photography and Media Club photography experience (May-July 2025), Phoenix Technical Society ECE coordination, RoboVerse event organisation based on supplied details, GSSoC 2026 contributor/mentee acceptance, and CyberGeek CTF 2026 participation where useful. Do not imply winning, ranking, or an award without evidence.

I. Education and certificates

Use the supplied AKGEC and Lions School dates. List Python (Basic), Introduction to Cybersecurity, Cybersecurity Fundamentals, Google Prompting Essentials Specialization, and Networking Basics. Link certificates only when their URLs can be verified.

J. Coding profiles

Link GitHub, LeetCode, HackerRank, LinkedIn, and Credly. Instagram can be an optional secondary link. Do not fetch unreliable stats or show broken third-party widgets. If profile statistics are unavailable, show the profile links.

K. Contact

Use priyansh.singh019@gmail.com, a working mailto link, copy-email button with success feedback, professional profiles, a final CTA, and a compact footer. Never fake a contact form. If a form is added, connect it to a real configured service with proper loading, error, success, and spam handling. Keep the mobile number out of the public UI unless enabled centrally.

9. Seed project data

Use one typed project-data source instead of hardcoding project details across components. Start with the inventory in Section 4 of this document and inspect repositories before expanding technical claims.

- **SentinelPy:** source, frontend, backend, API docs. Determine whether NetSecure was an earlier name for the same project.
- **ClockWork:** source, live app. AI everyday-task application; inspect current code.
- **NovaBank:** source, live app. Banking-themed project; do not imply production banking functionality without evidence.
- **LeetCode Solutions:** repository, profile. Ongoing C++ practice.
- **NovaVoice Calendar Bridge:** Flask/Google Calendar local prototype; no confirmed public demo URL.
- **AI Workforce Management:** in-progress competition project; verify modules in the codebase.
- **Bus Route Explorer:** concept for local/intercity buses, stops, route options, and fares; no confirmed app URL.
- **SAT-SA:** exploration related to SOC assessment analytics; show publicly only if active work can be demonstrated.

Do not include the gesture-related project anywhere. Do not resurrect it through automatic repository discovery.

10. Project content and statuses

Use clear status types such as Live/Deployed, In Progress, Local Prototype, and Concept/Exploration. Repository links must be accurate; live links must point to the correct deployment; broken deployments must be described honestly; missing URLs must remain absent. Do not duplicate projects under old working names. Prioritise genuinely demonstrable work over a large list of unfinished ideas.

11. Theme system

- Respect operating-system preference on first visit.
- Persist the visitor’s choice between sessions.
- Apply theme tokens early to prevent theme flash where feasible.
- Style cards, menus, forms, borders, icons, hover states, overlays, and modals correctly in both themes.
- Provide a clear accessible toggle, state indicator, and sufficient contrast.
- Test theme changes while a modal or mobile menu is open.

A theme switch is not complete merely because the page background changes.

12. Responsive design

Test at 320-375 px, 390-430 px, about 768 px, 1024-1440 px, and around 1920 px. Ensure no unintended horizontal scrolling, overlapping headings, clipped text, inaccessible controls, broken sticky navigation, hover-only essential actions, oversized media on mobile, undersized touch targets, long-URL overflow, or modal overflow.

Build deliberately for small screens rather than merely compressing the desktop layout.

13. Accessibility

Use semantic HTML, a skip-to-content link, logical heading hierarchy, keyboard-accessible navigation, visible focus indicators, accessible buttons and links, appropriate aria labels, status announcements where useful, adequate contrast, correct overlay focus management, meaningful alt text, and `prefers-reduced-motion` support. Never communicate a status through colour alone. The site must work by keyboard and non-hover input.

14. Performance and stability

- Optimise images and lazy-load below-the-fold media.
- Keep the hero lightweight and avoid unnecessary dependencies.
- Avoid expensive state updates on every animation frame and layout thrashing.
- Prevent layout shifts and clean up listeners, timers, animation frames, and observers.
- Handle failed images and unavailable APIs gracefully.
- Make the site progressive: it should remain attractive if animation, external data, or a deployment is unavailable.
- Do not autoplay audio.

Aim for strong Lighthouse performance, accessibility, best-practice, and SEO scores. Measure these when tools are available; do not claim a score without testing.

15. SEO and professional presentation

Add a meaningful title, concise meta description, canonical URL configuration only when the real domain is known, Open Graph and social-sharing metadata, favicon/original wordmark, correct headings, useful link labels, and a suitable page description. Do not invent a domain or claim a deployment is complete. A CV download must point to a real PDF asset such as `/resume.pdf`; do not show the button if the file is absent.

16. Code architecture

Use a maintainable structure, adapted to the existing repository. A suggested structure is:

```
src/
  components/
    navigation/  hero/  effects/  projects/
    about/  experience/  skills/  certifications/
    contact/  footer/  ui/
  data/portfolio.ts
  hooks/
  styles/
  types/
  utils/
```

Use strong project/profile types, reusable UI components, centralised theme tokens and external links, predictable state, clear presentation/content separation, and consistent error handling. Avoid one enormous component for the entire site and do not invent fake API calls.

17. Actual functionality

Test logo navigation; main section links; mobile menu; theme switching/persistence; project filtering; project details; Escape-key closing; source/demo/profile links; email and copy-email; resume download if present; scroll progress; active navigation; cursor interactions; responsive menus; reduced motion; and browser Back/Forward behaviour if routes or modal navigation require it.

Do not leave placeholder buttons, empty links, fake loading states, or unexplained disabled controls.

18. Testing and bug fixing

Run all supported checks: dependency installation, linting, type checking, production build, unit tests, end-to-end tests where available, link verification, responsive checks, browser console checks, and failed-network-request inspection.

If browser automation is available, launch the app and test desktop, mobile, both themes, main sections, project details, keyboard navigation, reduced motion, and console output. Pay special attention to React keys, state errors, missing assets, imports, broken URLs, route handling, overflow, leaked cursor events, animation loops, theme flash, and keyboard focus.

Do not claim a test passed unless it was actually run and passed. Fix reproducible issues and repeat relevant checks.

19. Definition of done

- Distinctive visual identity with a smooth, non-blocking liquid effect.
- Reliable custom cursor on eligible devices.
- Both themes feel complete.
- Correct responsiveness from small mobile to large desktop.
- Accurate project content and status labels.
- Legitimate destinations for every link.
- The gesture-related project is absent.
- No known broken interactions remain.
- Accessible alternatives exist for animation and hover effects.
- Production build succeeds and relevant tests are run.
- Remaining issues are documented honestly.

20. Required final report from the coding agent

After implementation, provide a factual summary of the main features, key files created/modified, actual stack, run commands, build/lint/type-check/test results, links verified, unverified deployments or credentials, and remaining issues that prevent readiness.

Do not return only a plan, design explanation, or disconnected snippets. Build the working application in the current repository, verify what is possible, and fix the issues found.

Final direction
The result should feel fluid, intelligent, personal, technically credible, and ready to demonstrate at an in-person hackathon. The animation should attract attention initially, then stay out of the way while judges evaluate the projects.

6. Before the Hackathon

1. Open each published project and confirm that the deployment loads. If it does not, fix it or label the deployment status honestly.
2. Open every GitHub repository and verify that the portfolio uses the correct name, README, technologies, and source URL.
3. Check all profiles and credential links from the portfolio itself.
4. Test on the laptop and on a real mobile device, including the light and dark themes.
5. Check keyboard navigation, reduced motion, cursor interactions, project modals, copy-email, and the mobile menu.
6. Run the production build and the available tests after the final visual changes.
7. Deploy the portfolio and test the deployed URL, not just localhost.
8. Prepare a short explanation of the architecture and trade-offs for each featured project.

> **Important:** A prompt cannot guarantee zero bugs. A credible “ready for demo” claim comes from running the application, validating the links, testing real interactions, and fixing reproducible issues.

Profile and project information in this brief reflects the uploaded CV and previously discussed/publicly linked repositories. Recheck external links immediately before the hackathon because deployment availability can change.